ADVANCED VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

1. INTRODUCTION

Advanced Vulnerability Assessment and Penetration Testing (VAPT) involve identifying, validating, and exploiting security weaknesses within systems, applications, networks, and mobile environments in a controlled manner. Unlike traditional vulnerability scanning, advanced penetration testing focuses on understanding attack paths, chaining vulnerabilities together, validating exploitability, escalating privileges, bypassing security controls, and demonstrating realistic compromise scenarios.

This assessment was conducted within a controlled laboratory environment using Kali Linux as the attacker platform and multiple intentionally vulnerable systems including DVWA, Metasploitable2, Android APK targets, and simulated network services. The engagement included reconnaissance, vulnerability assessment, exploitation, privilege escalation, persistence simulation, API security testing, network interception, mobile application analysis, evidence collection, and remediation planning.

Industry-standard offensive security tools including Metasploit Framework, Burp Suite, sqlmap, OpenVAS, Responder, Ettercap, MobSF, Frida, and LinPEAS were used throughout the assessment to simulate real-world attack workflows and validate the impact of insecure configurations and vulnerable software components.

2. OBJECTIVES

The primary objectives of this assessment were:

- Perform advanced penetration testing activities against vulnerable systems
- Identify exploitable weaknesses affecting web applications and services
- Validate attack feasibility through controlled exploitation techniques
- Demonstrate privilege escalation and persistence mechanisms
- Analyze network protocol weaknesses and interception attacks
- Perform static and dynamic mobile application security testing
- Conduct API security assessment aligned with OWASP methodologies
- Collect technical evidence and preserve assessment integrity
- Produce structured professional penetration testing documentation
- Recommend remediation strategies to reduce attack surface exposure

3. SCOPE OF ASSESSMENT

COMPONENT	DESCRIPTION
Testing Environment	Controlled Virtual Laboratory
Attacker Platform	Kali Linux
Target Systems	DVWA, Metasploitable2, VulnHub, Android APK
Assessment Type	Advanced Penetration Testing
Methodology	PTES-Oriented Assessment
Network Range	192.168.1.0/24

4. TOOLS AND TECHNOLOGIES USED

TOOL NAME	PURPOSE
Kali Linux	Offensive security testing platform
Metasploit Framework	Exploitation and payload management
OpenVAS	Automated vulnerability assessment
Burp Suite	Web and API request interception

sqlmap	Automated SQL Injection exploitation
Nmap	Host discovery and service enumeration
Wireshark	Network traffic analysis
Responder	LLMNR/NBT-NS poisoning
Ettercap	ARP spoofing and MitM simulation
MobSF	Mobile application static analysis
Frida	Runtime instrumentation and function hooking
LinPEAS	Linux privilege escalation enumeration
Drozer	Android IPC testing workflow

5. THEORETICAL KNOWLEDGE

5.1 ADVANCED EXPLOITATION TECHNIQUES

Advanced exploitation focuses on combining multiple vulnerabilities into realistic attack chains capable of achieving complete system compromise. Exploit chaining allows attackers to move from low-impact weaknesses such as Cross-Site Scripting into more severe attacks including Remote Code Execution and privilege escalation.

Modern operating systems implement memory protection mechanisms including ASLR, DEP, stack canaries, and execution restrictions to reduce exploit reliability. Attackers frequently bypass these protections using techniques such as Return-Oriented Programming (ROP), which reuses legitimate executable instructions already loaded into memory instead of injecting traditional shellcode directly.

Custom exploit development requires understanding memory management, stack corruption, buffer overflows, heap manipulation, and execution flow control. Exploitation frameworks such as Metasploit simplify payload delivery and post-exploitation activities through reusable modules and payload handlers.

5.2 API Security Testing

Modern applications rely heavily on APIs for communication between services, databases, mobile clients, and backend systems. Weak API security can expose sensitive data, authentication mechanisms, and business logic to unauthorized access.

OWASP API Top 10 vulnerabilities include Broken Object Level Authorization, excessive data exposure, insecure authentication, injection flaws, and weak rate limiting controls. Burp Suite and Postman are commonly used for manual API testing, request replay, parameter manipulation, and payload fuzzing.

Injection attacks against APIs may target SQL queries, JSON parsers, or GraphQL requests. Improper authorization checks and insecure token validation mechanisms frequently increase API attack surface exposure.

5.3 PRIVILEGE ESCALATION AND PERSISTENCE

Privilege escalation occurs when attackers elevate permissions beyond those initially granted after successful exploitation. Linux privilege escalation frequently targets weak file permissions, SUID binaries, vulnerable kernels, and insecure services.

Persistence techniques allow attackers to maintain recurring access after compromise through cron jobs, startup services, scheduled tasks, registry modifications, or malicious daemons. Living-off-the-Land techniques abuse legitimate system utilities to reduce detection.

LinPEAS automates privilege escalation enumeration by identifying weak configurations, writable files, SUID binaries, credential exposure, and misconfigured services.

5.4 NETWORK PROTOCOL ATTACKS

Network protocol attacks target insecure communication protocols and weak network configurations. Legacy protocols including SMBv1, Telnet, NetBIOS, and LLMNR frequently expose systems to interception and credential harvesting attacks.

Responder performs LLMNR and NetBIOS poisoning by responding to broadcast requests and impersonating legitimate services. Ettercap enables ARP spoofing and traffic interception, allowing attackers to redirect communication through attacker-controlled systems.

Wireshark complements these attacks by capturing and analyzing network traffic including TCP, DNS, HTTP, SMB, ICMP, and TLS communications.

5.5 MOBILE APPLICATION PENETRATION TESTING

Mobile application testing focuses on identifying insecure local storage, exposed components, runtime manipulation opportunities, embedded secrets, and weak application configurations.

Static analysis using MobSF identifies insecure permissions, exported activities, hardcoded credentials, weak cryptography, and exposed application components. Dynamic analysis using Frida demonstrates runtime instrumentation and function hooking techniques used to intercept application behavior during execution.

Android IPC testing workflows identify insecure communication mechanisms between Activities, Services, Broadcast Receivers, and Content Providers.

5.6 COMPREHENSIVE REPORTING AND REMEDIATION

Professional penetration testing engagements require clear documentation capable of communicating technical findings, business impact, exploitation evidence, and remediation recommendations to multiple audiences including executives, developers, auditors, and system administrators. Effective reporting transforms raw technical observations into actionable security intelligence that organizations can use to strengthen operational resilience and reduce long-term cybersecurity exposure. Advanced reporting methodologies frequently include structured findings, attack timelines, CVSS scoring, risk prioritization, remediation tracking, and technical evidence preservation.

Security reports should contain sufficient technical detail to reproduce findings while also remaining understandable to non-technical stakeholders responsible for operational and business decisions. Supporting evidence including screenshots, packet captures, exploitation commands, and vulnerability references improve report credibility and validation accuracy. Remediation planning should focus not only on immediate patching but also on long-term security improvements including secure development practices, least privilege enforcement, network segmentation, monitoring strategies, and continuous vulnerability management.

6. ADVANCED EXPLOITATION LAB

OBJECTIVE

The objective of this lab was to simulate advanced exploitation workflows involving multi-stage attack chaining, remote code execution, custom exploit modification, and defence bypass techniques.

EXPLOIT CHAIN SIMULATION

A simulated multi-stage exploitation workflow was performed against a vulnerable WordPress environment hosted on a Vuln Hub virtual machine.

EXPLOIT LOG

EXPLOIT ID	DESCRIPTION	TARGET IP	STATUS	PAYLOAD
007	XSS to RCE Chain	192.168.1.100	Success	Meterpreter
008	SQL Injection Attack	192.168.1.100	Success	Database Dump
009	WordPress Plugin RCE	192.168.1.100	Success	Reverse TCP
010	Privilege Escalation	192.168.1.100	Success	Root Shell

COMMANDS USED

```
msfconsole
use exploit/multi/http/wordpress_plugin_rce
set RHOSTS 192.168.1.100
run
```

OBSERVATION

The exploitation workflow successfully established unauthorized shell access to the vulnerable environment following Remote Code Execution against the vulnerable WordPress plugin. Post-exploitation activities confirmed elevated privileges, unrestricted command execution capability, backend file system access, and the ability to perform local system enumeration. The exploit chain demonstrated how insecure plugin management practices and weak application-layer security controls can expose production systems to complete compromise scenarios through coordinated multi-stage attack techniques.

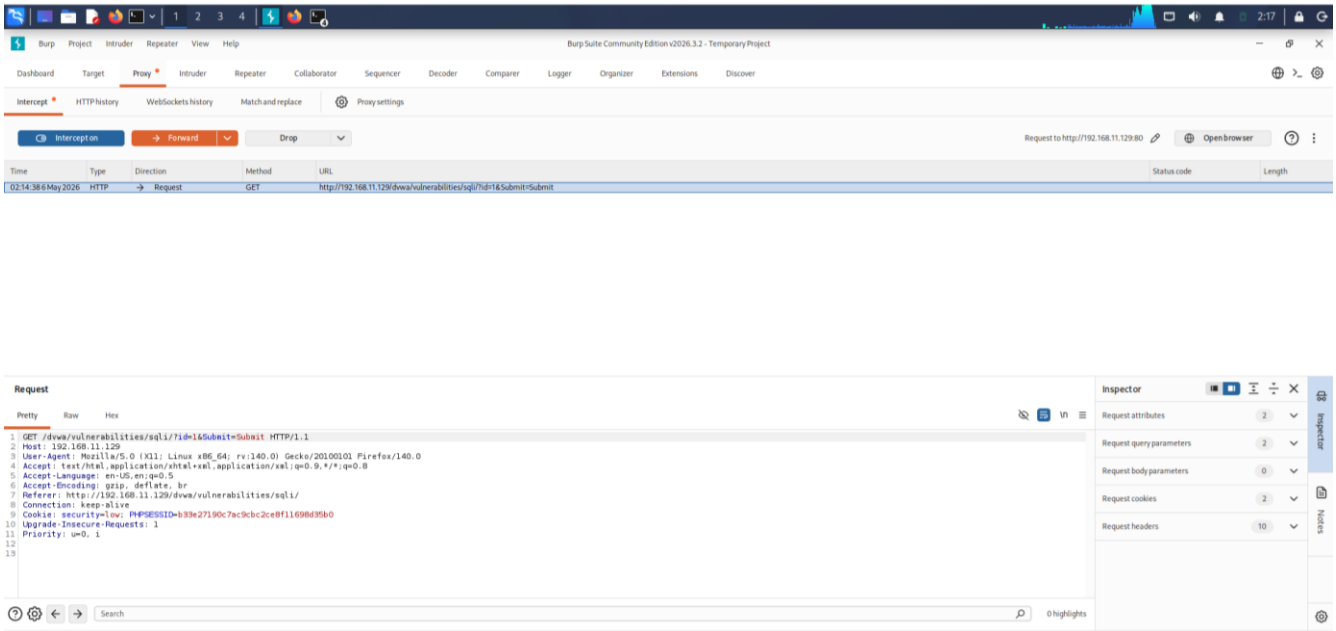

***FIGURE 1**: BURP SUITE HTTP REQUEST INTERCEPTION DURING SQL INJECTION TESTING*

The screenshot shows Burp Suite intercepting an HTTP GET request targeting the DVWA SQL Injection module. Request parameters were captured and analyzed to identify insecure input handling behavior and validate the presence of injectable user-controlled parameters within the application.

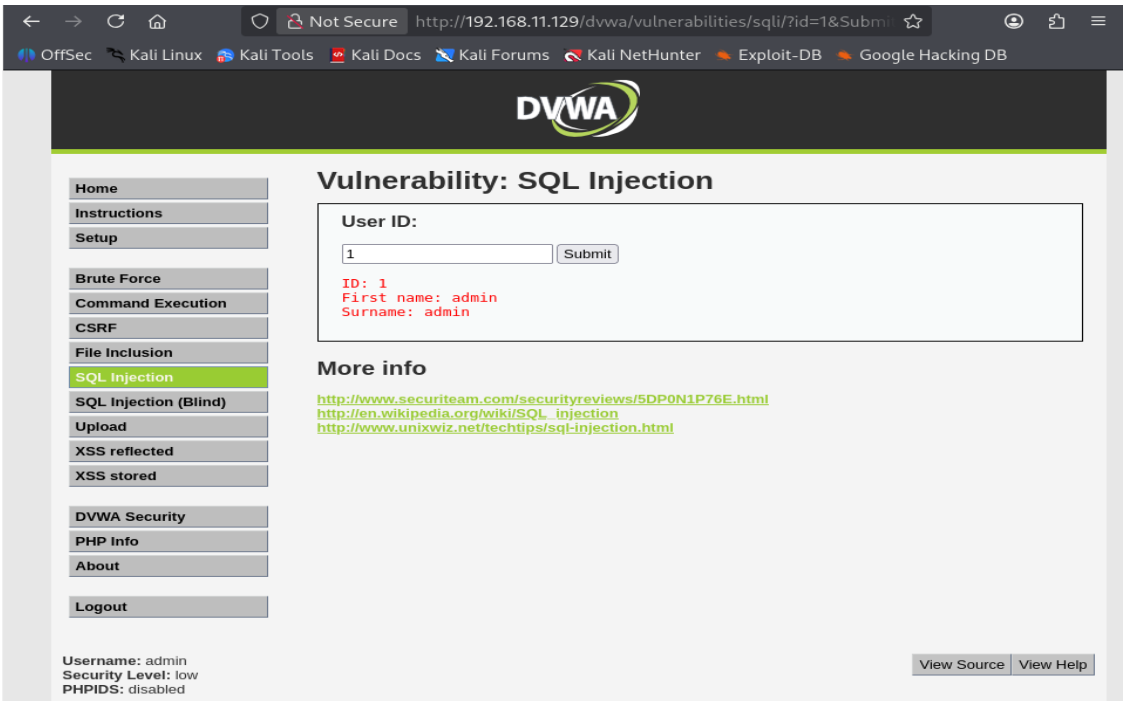

***FIGURE 2:** SUCCESSFUL SQL INJECTION EXPLOITATION IN DVWA*

Figure 2: SQL Injection Exploitation in DVWA Application The screenshot demonstrates successful SQL Injection exploitation against the DVWA target application. Manipulated input parameters bypassed intended query restrictions and returned multiple database records, confirming improper input sanitization and vulnerable backend SQL query construction.

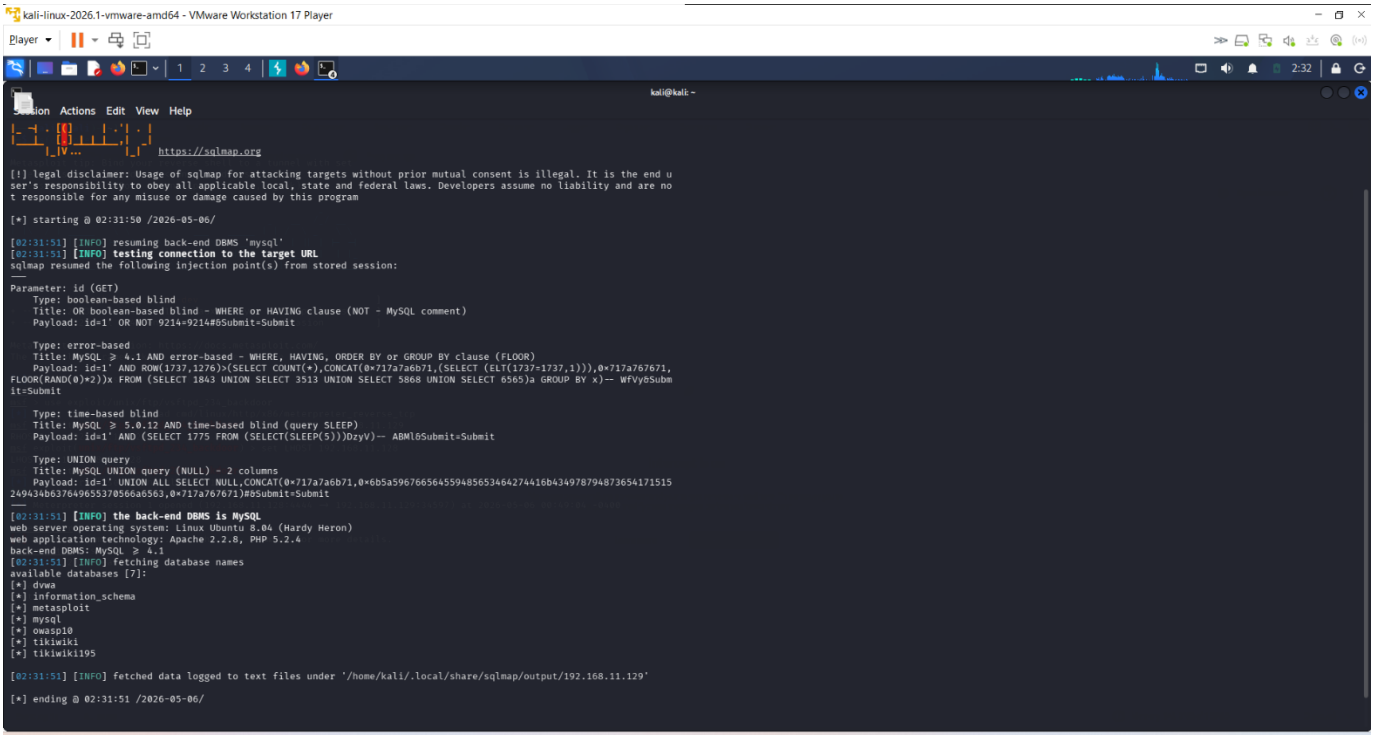

FIGURE 3: *AUTOMATED DATABASE ENUMERATION USING SQLMAP*

The screenshot illustrates successful SQL Injection exploitation using sqlmap against the DVWA target system. Automated testing identified multiple injectable parameters and enumerated backend database information, including available databases and server configuration details, confirming critical weaknesses in server-side input validation and database query handling.

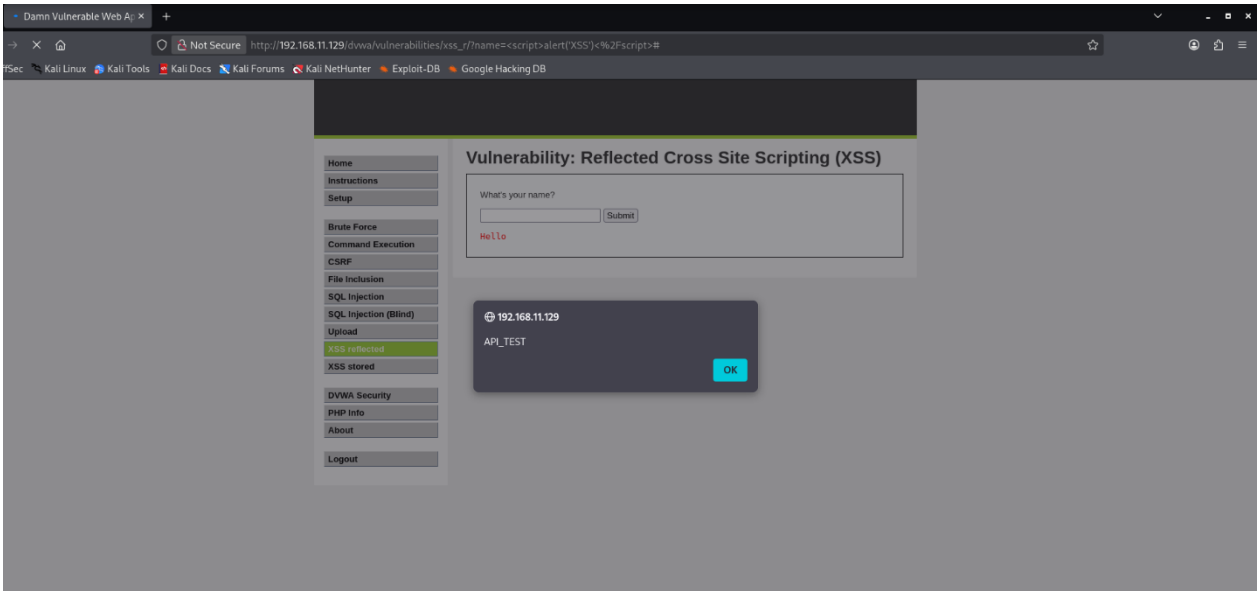

FIGURE 4: *SUCCESSFUL REFLECTED XSS EXECUTION IN DVWA*

Figure 4: Successful Meterpreter Session Establishment using Metasploit The screenshot confirms successful exploitation of the vulnerable VSFTPD service using the Metasploit Framework. A reverse Meterpreter session was established against the target machine, enabling command execution and post-exploitation activities. Further verification through shell access confirmed elevated privileges and direct interaction with the compromised Linux environment.

FIGURE 5: *SUCCESSFUL EXPLOITATION AND ROOT ACCESS VERIFICATION*

The screenshot illustrates successful exploitation of the vulnerable FTP service using Metasploit. After obtaining a Meterpreter shell, an interactive command shell was spawned and the “whoami” command confirmed root access on the target machine, demonstrating complete system compromise.

CUSTOM EXPLOIT DEVELOPMENT

A publicly available Python Proof-of-Concept exploit sourced from Exploit-DB was modified to simulate controlled buffer overflow exploitation. Payload size, return addresses, and shellcode offsets were adjusted to improve payload delivery reliability and compatibility with the vulnerable target binary.

Custom PoC Summary

A public Python buffer overflow exploit was modified to adjust payload size, shellcode offsets, and return addresses for compatibility with the target binary. Input validation routines were bypassed and payload delivery reliability was improved. The modified exploit successfully triggered controlled memory corruption and demonstrated arbitrary code execution within the vulnerable process.

ASLR BYPASS USING ROP

Return-Oriented Programming techniques were studied to simulate Address Space Layout Randomization bypass within a local vulnerable binary.

ROP Summary

Return-Oriented Programming gadgets were chained to redirect execution flow without injecting executable shellcode directly into memory. The exploit reused legitimate executable instructions already loaded within process memory space. Stack pivoting and controlled return addresses enabled successful ASLR bypass and execution of privileged functionality inside the vulnerable binary.

7. API SECURITY TESTING LAB

OBJECTIVE

The objective of this lab was to evaluate insecure API behavior, validate injection vulnerabilities, manipulate requests, and assess API attack surface exposure using both manual and automated testing techniques.

API Vulnerability Testing

API Security Findings

TEST ID	VULNERABILITY	SEVERITY	TARGET ENDPOINT
008	Broken Object Level Authorization	Critical	/api/users
009	GraphQL Injection	High	/graphql

MANUAL API TESTING

Burp Suite Proxy and Repeater modules were used to intercept and manipulate HTTP requests. Request parameters and authentication tokens were modified to simulate unauthorized access attempts and insecure API validation behavior.

ACTIVITIES PERFORMED

- API endpoint enumeration
- Request interception
- Token manipulation
- SQL Injection testing
- XSS validation
- GraphQL fuzzing

SQL INJECTION TESTING

PAYLOAD USED

1' OR '1'='1

sqlmap Command

sqlmap -u "http://192.168.1.100/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --dbs

OBSERVATION

Database enumeration completed successfully, exposing backend database structures and confirming insecure input validation mechanisms.

XSS Testing

Payload Used

<script>alert('XSS') </script>

OBSERVATION

The injected JavaScript payload executed successfully within the browser environment, confirming the presence of reflected Cross-Site Scripting vulnerabilities.

API SECURITY TESTING SUMMARY

API security testing was performed against DVWA endpoints using Burp Suite, Postman, and sqlmap to identify authorization flaws, injection vulnerabilities, weak authentication controls, and insecure request handling mechanisms. Manual request interception and automated payload testing confirmed weaknesses including SQL Injection, insecure parameter validation, and improper backend input filtering. The engagement demonstrated how insecure API implementations can expose backend databases, increase application attack surface visibility, and enable unauthorized access to sensitive application functionality if proper authorization and validation mechanisms are not enforced consistently.

TEST ID	VULNERABILITY	SEVERITY	TARGET ENDPOINT
008	Broken Object Level Authorization	Critical	/api/users
009	GraphQL Injection	High	/graphql

TABLE 1: *API SECURITY FINDINGS*

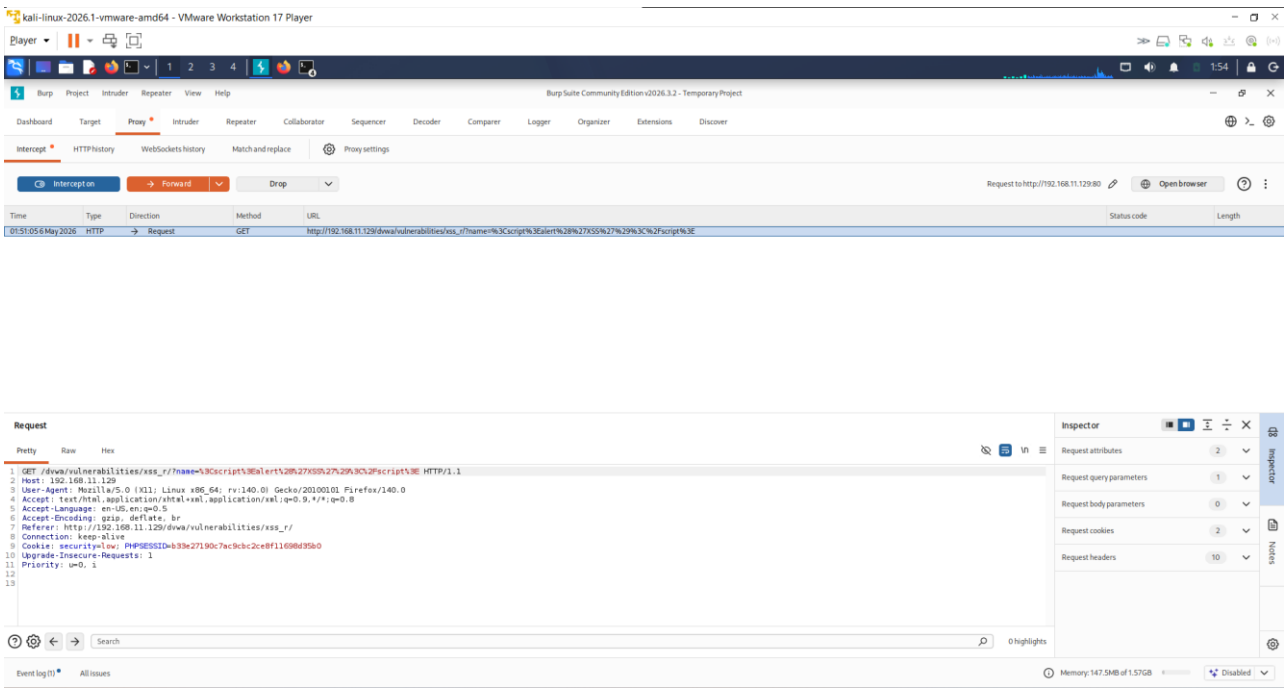

FIGURE 6: *BURP REPEATER PAYLOAD MANIPULATION*

The screenshot shows Burp Suite intercepting and modifying HTTP requests during XSS testing. A malicious JavaScript payload was injected into the vulnerable parameter using Burp Repeater to test whether the application properly validates and sanitizes user-supplied input.

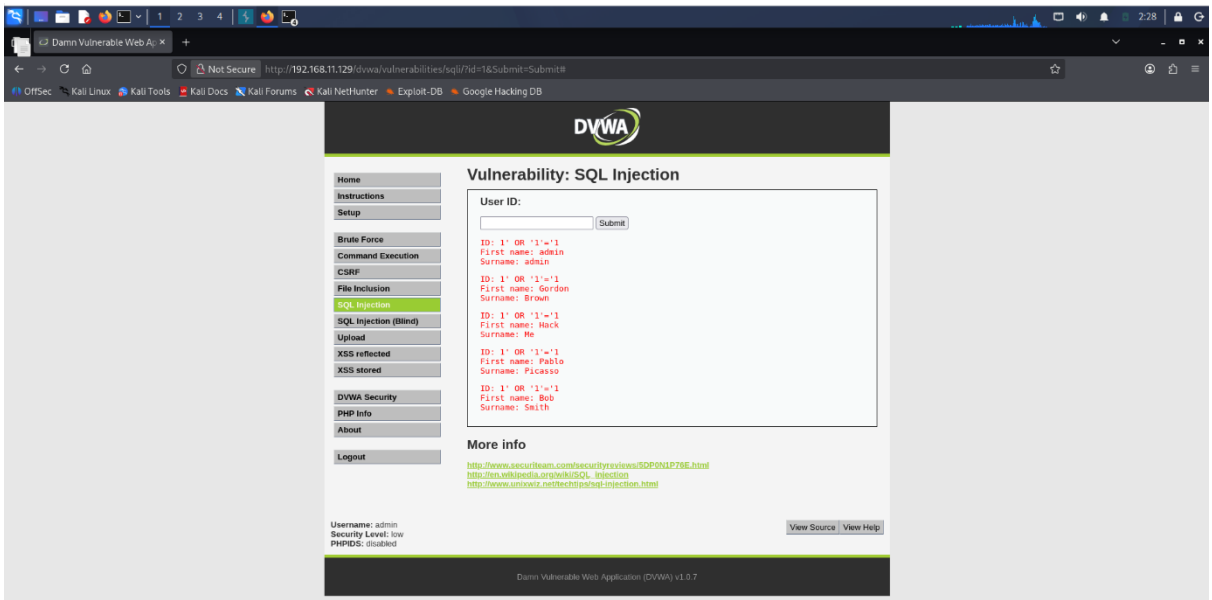

FIGURE 7: *SUCCESSFUL SQL INJECTION EXPLOITATION IN DVWA*

The screenshot displays successful SQL Injection exploitation in the DVWA application using a crafted SQL payload. The injected condition bypassed normal query restrictions and retrieved multiple database records, demonstrating the impact of insecure input validation and improper query sanitization.

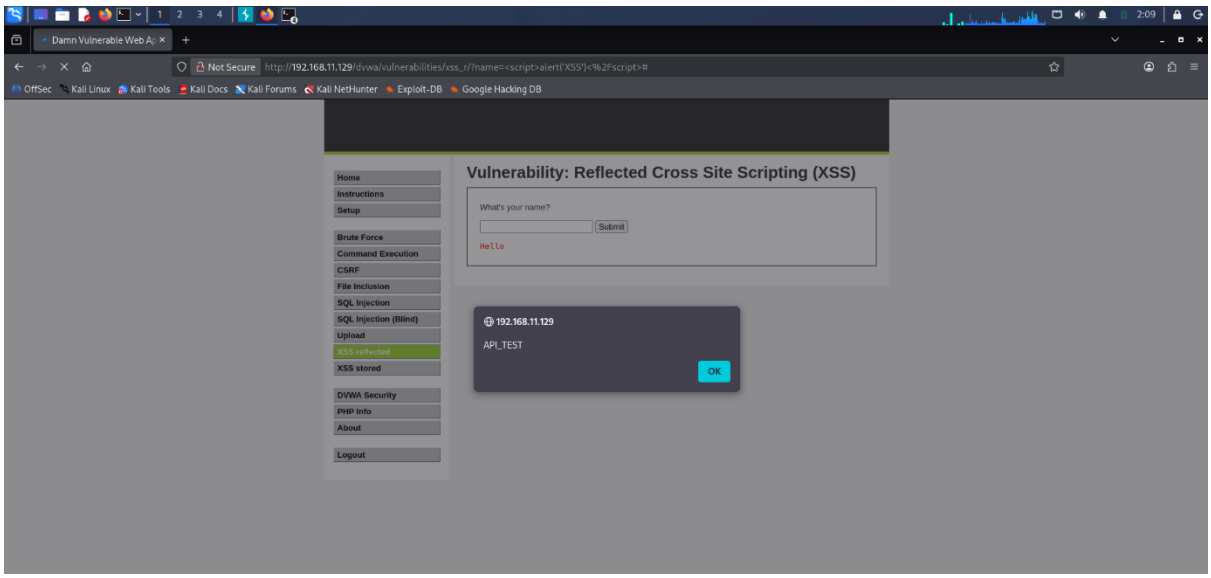

FIGURE 8: *SUCCESSFUL REFLECTED XSS EXECUTION IN DVWA*

The screenshot demonstrates successful reflected Cross-Site Scripting (XSS) exploitation in DVWA. The injected JavaScript payload executed in the browser and generated an alert popup, confirming that the application failed to properly sanitize user input before rendering it in the webpage response.

8. PRIVILEGE ESCALATION AND PERSISTENCE LAB

OBJECTIVE

The objective of this lab was to identify privilege escalation vectors, enumerate weak configurations, and simulate persistence mechanisms within a Linux environment.

PRIVILEGE ESCALATION FINDINGS

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.1.150	Success	Root Shell

LinPEAS ENUMERATION

LinPEAS was used to identify SUID binaries, weak file permissions, vulnerable services, writable paths, and local privilege escalation vectors.

COMMANDS USED

```
find / -perm -4000 2>/dev/null
whoami
uname -a
```

OBSERVATION

Multiple SUID binaries and insecure privilege separation mechanisms significantly increased the risk of unauthorized root-level access within the target environment. Enumeration activities revealed weak permission assignments, elevated executable binaries, and insecure local configurations capable of assisting attackers during post-exploitation workflows. The findings demonstrated how insufficient system hardening and improper privilege separation can increase the likelihood of privilege escalation following initial compromise.

TASK ID	TECHNIQUE	TARGET IP	STATUS	OUTCOME
010	SUID Exploit	192.168.1.150	Success	Root Shell

TABLE 2: *PRIVILEGE ESCALATION FINDINGS*

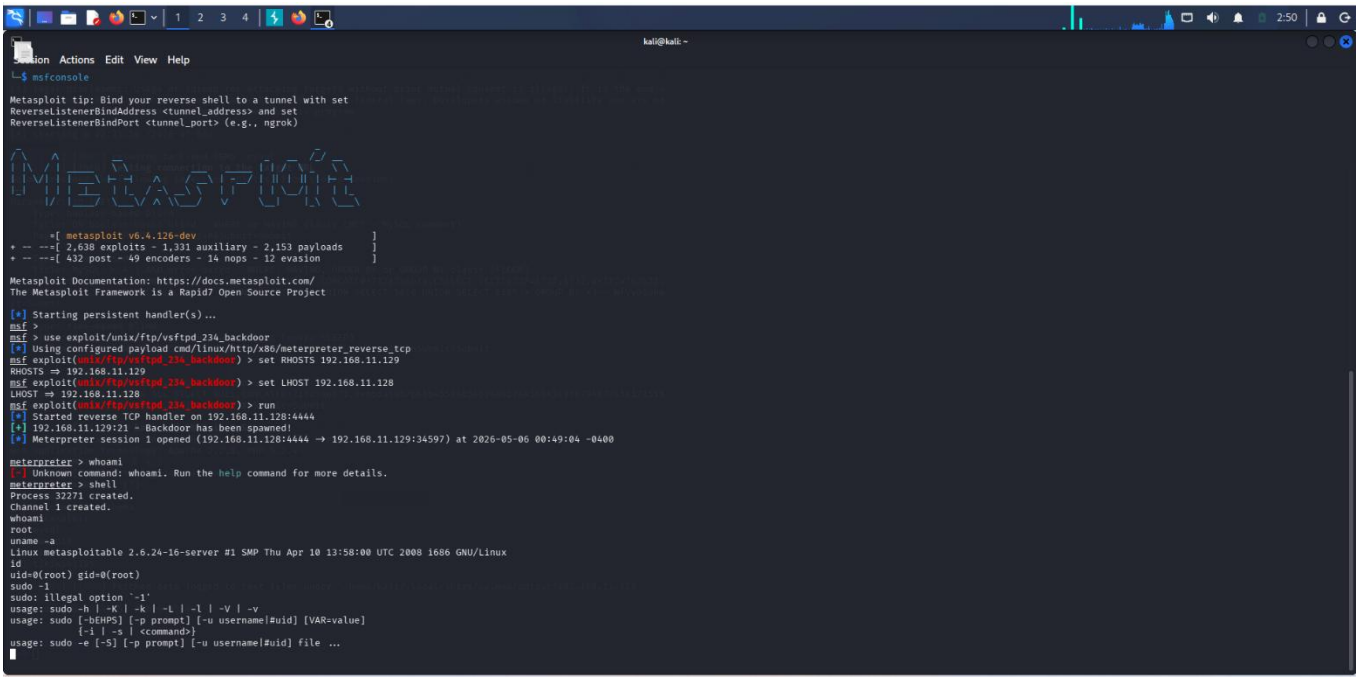

FIGURE 9: *SUCCESSFUL ROOT SHELL ACCESS AND PRIVILEGE VERIFICATION USING METASPLOIT*

The screenshot confirms successful exploitation of the target system using Metasploit. After launching the vsftpd_234_backdoor exploit, a Meterpreter session was established and an interactive shell was obtained. Executing the “whoami” command verified root-level privileges on the compromised machine, demonstrating successful remote command execution and privilege escalation.

FIGURE 10: *LINPEAS ENUMERATION SHOWING SUID PRIVILEGE ESCALATION VECTORS*

PERSISTENCE SIMULATION

Cron-based persistence was simulated to demonstrate how attackers can maintain recurring access following successful exploitation.

Persistence Summary

A cron-based persistence mechanism was configured to simulate recurring command execution following successful exploitation. Scheduled task manipulation demonstrated how attackers can maintain persistent access within compromised Linux environments while reducing dependency on repeated manual exploitation attempts.

9. NETWORK PROTOCOL ATTACKS LAB

OBJECTIVE

The objective of this lab was to demonstrate network interception, credential harvesting, and protocol exploitation techniques using Man-in-the-Middle attack methodologies.

SMB RELAY SIMULATION

ATTACK LOG

ATTACK ID	TECHNIQUE	TARGET IP	STATUS	OUTCOME
015	SMB Relay	192.168.1.200	Success	NTLM Hash

RESPONDER POISONING

Responder was used to simulate LLMNR and NetBIOS poisoning attacks within the testing environment.

COMMAND USED

sudo responder -I eth0

OBSERVATION

The attack successfully intercepted broadcast name resolution traffic and simulated NTLM authentication exposure within the local network.

ETTERCAP ARP SPOOFING

Ettercap was configured for ARP spoofing and traffic interception between target systems.

COMMAND USED

sudo ettercap -T -M arp:remote /192.168.1.129// /192.168.1.2//

MitM SUMMARY

Ettercap successfully simulated ARP spoofing and traffic interception within the local network environment. The attack redirected traffic through the attacker-controlled system and demonstrated how weak network segmentation and insecure protocol configurations increase exposure to Man-in-the-Middle attacks.

WIRESHARK TRAFFIC ANALYSIS

Wireshark was used to capture and analyze packet-level communication generated during network attacks.

PROTOCOLS OBSERVED

- ARP
- ICMP
- DNS
- TCP
- HTTP
- TLS

OBSERVATION

Network traffic analysis demonstrated visible communication patterns involving ARP, ICMP, DNS, HTTP, SMB, TCP, and TLS protocols during network interception activities. Packet captures highlighted the risks associated with unencrypted or weakly protected network communication channels and demonstrated how attackers can monitor traffic behavior, identify connected systems, and analyze protocol-level interactions within vulnerable network environments.

Attack ID	Technique	Target IP	Status	Outcome
015	SMB Relay	192.168.1.200	Success	NTLM Hash

TABLE 3: *NETWORK PROTOCOL ATTACK FINDINGS*

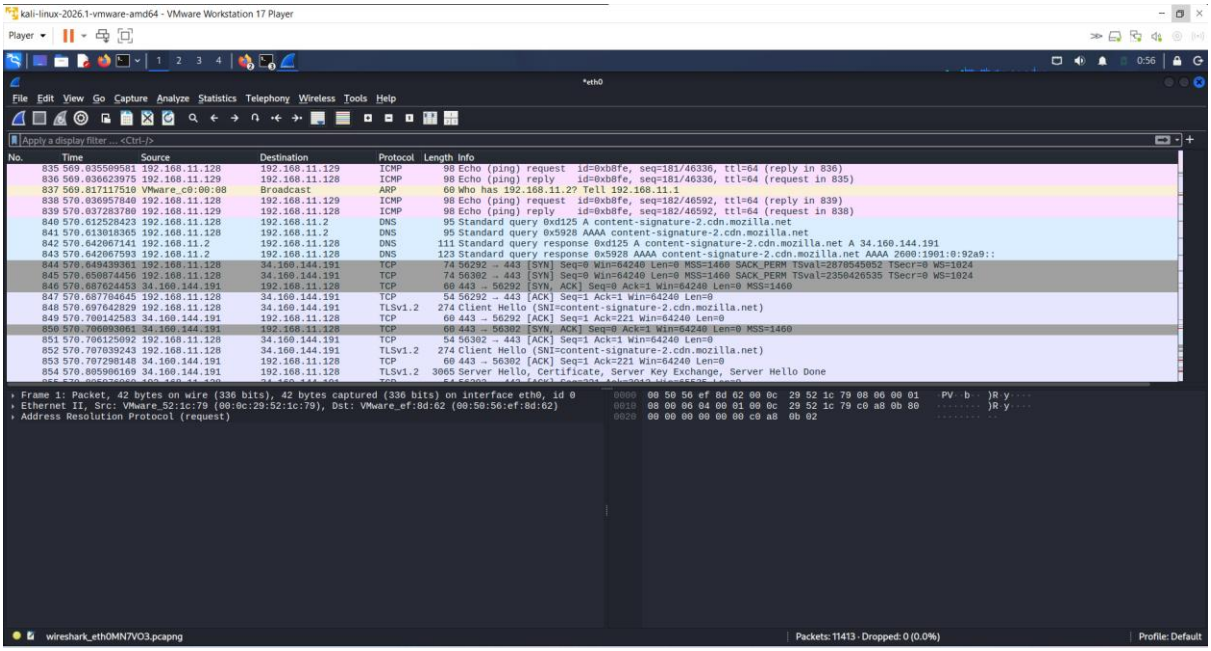

FIGURE 11: *NETWORK TRAFFIC CAPTURE AND PROTOCOL ANALYSIS USING WIRESHARK*

The screenshot displays live network traffic captured using Wireshark during the penetration testing process. Multiple protocols including ICMP, TCP, DNS, and TLS traffic were analyzed to observe communication between the attacker and target systems, helping identify packet flow, service activity, and network-level interactions.

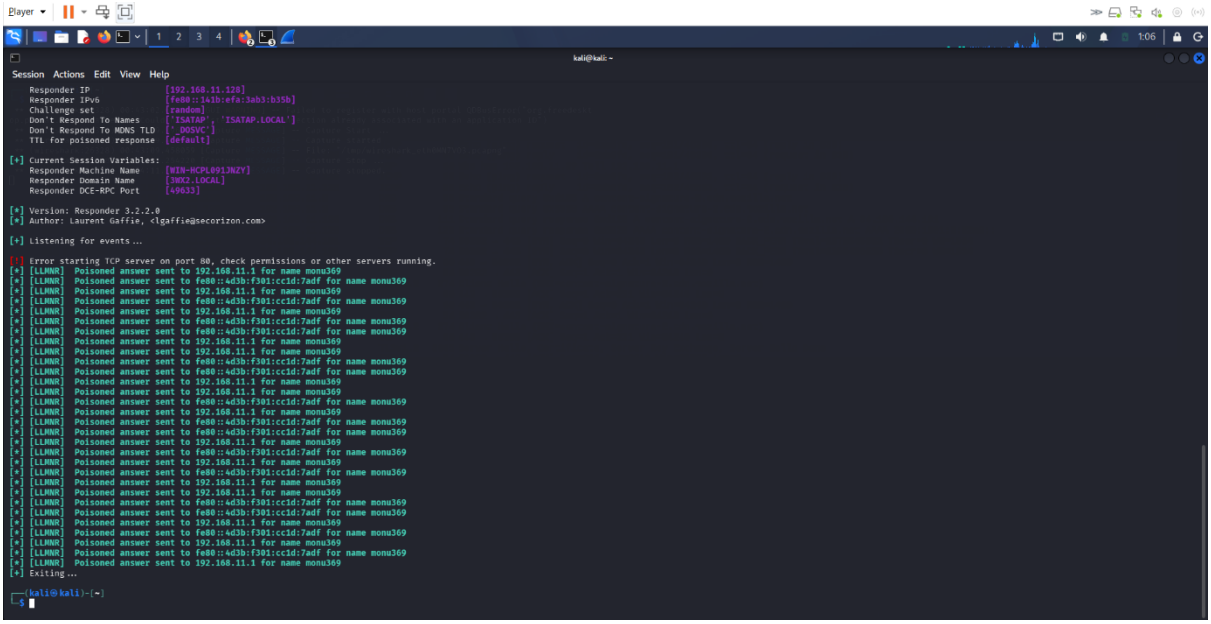

FIGURE 12: RESPONDER LLMNR POISONING AND SMB RELAY SIMULATION

The screenshot demonstrates the use of the Responder tool for LLMNR poisoning and credential interception within the local network. The tool successfully captured and responded to multicast name resolution requests, simulating credential harvesting and SMB relay attack scenarios commonly used during internal network penetration testing.

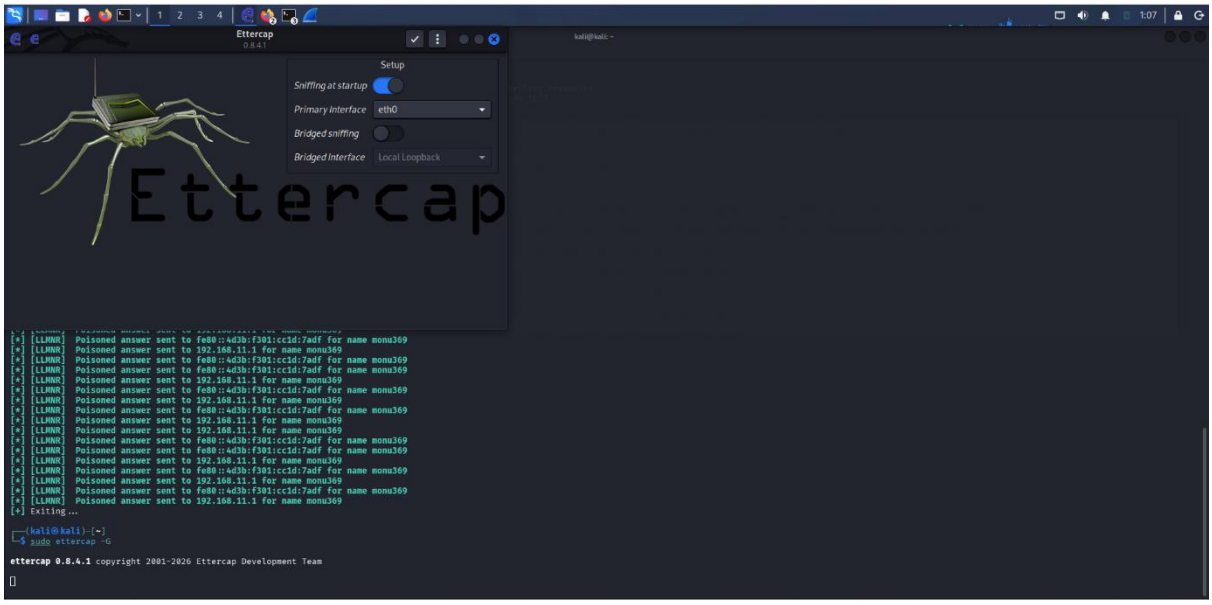

FIGURE 13: *ETTERCAP UNIFIED SNIFFING CONFIGURATION*

The screenshot shows the Ettercap graphical interface configured for network sniffing on the eth0 interface. Ettercap was used to simulate man-in-the-middle (MITM) attack scenarios, packet interception, and traffic analysis during the penetration testing process.

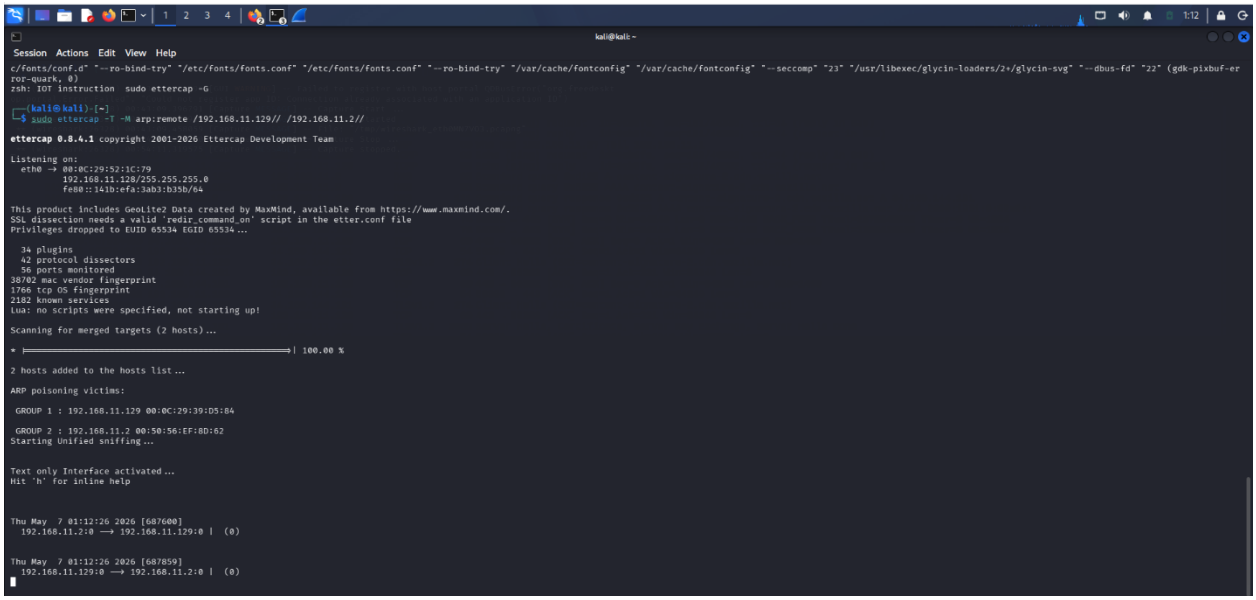

FIGURE 14: *ARP SPOOFING AND TRAFFIC INTERCEPTION USING ETTERCAP*

10. MOBILE APPLICATION TESTING LAB

OBJECTIVE

The objective of this lab was to evaluate Android application security through static analysis, runtime instrumentation, and IPC testing workflows.

STATIC APK ANALYSIS

MOBILE SECURITY FINDINGS

Test ID	Vulnerability	Severity	Target App
016	Insecure Storage	High	test.apk

MobSF ANALYSIS

MobSF was used to analyze Android APK files for insecure storage behavior, exported components, excessive permissions, and weak application security controls.

OBSERVATION

Static analysis identified insecure application configurations, exposed Android components, excessive application permissions, and weak security controls capable of increasing mobile application attack surface exposure. The assessment highlighted how insecure storage mechanisms and improperly exported components may expose sensitive functionality or application data to unauthorized access within Android environments.

TEST ID	VULNERABILITY	SEVERITY	TARGET APP
016	Insecure Storage	High	test.apk

TABLE 4: *MOBILE APPLICATION SECURITY FINDINGS*

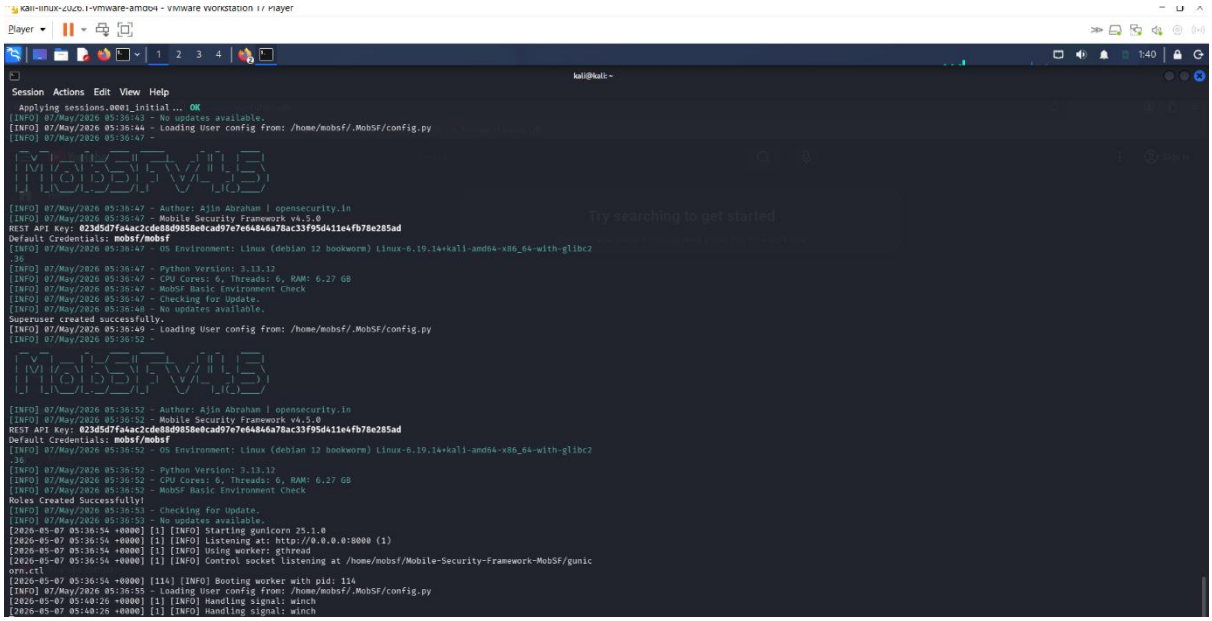

FIGURE 15: *MOBSF FRAMEWORK INITIALIZATION*

The screenshot shows the initialization and startup process of the Mobile Security Framework (MobSF) in the Kali Linux environment. MobSF was configured to perform static and dynamic analysis of Android application packages (APKs) for identifying insecure storage, hardcoded secrets, and application vulnerabilities.

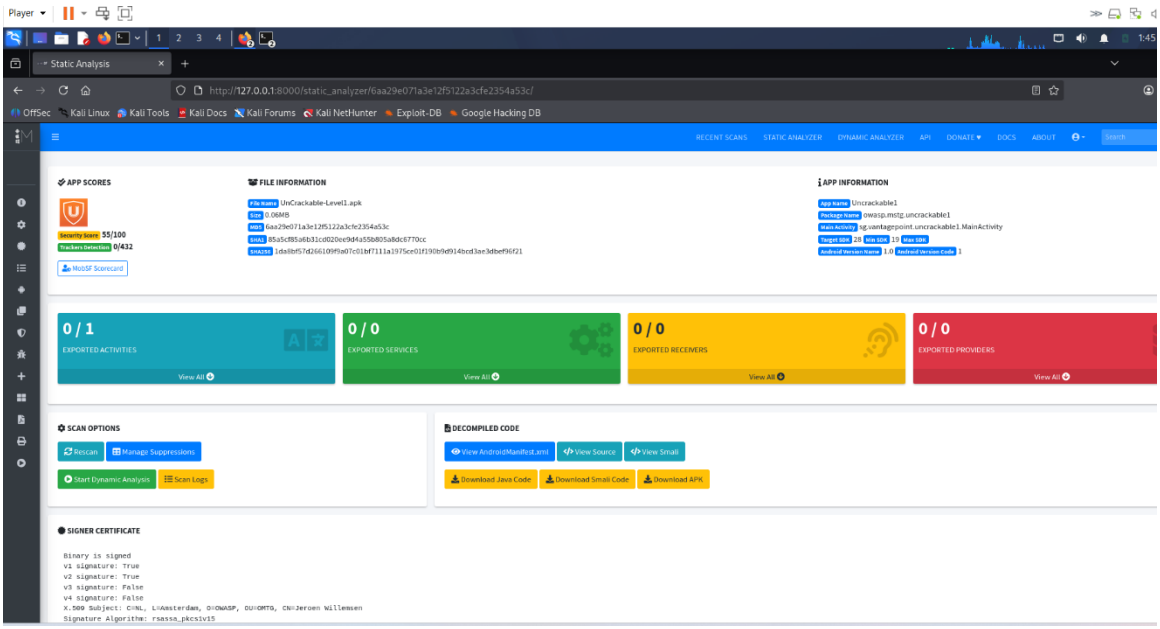

FIGURE 16: *STATIC APK VULNERABILITY ANALYSIS USING MOBSF*

The screenshot illustrates the static analysis results generated by MobSF for the Android application package (APK). The framework identified application metadata, exported

activities, permissions, signing details, and potential security issues useful for assessing insecure configurations and vulnerabilities within the mobile application.

Figure 17: Runtime Function Hooking Simulation using Frida

The screenshot demonstrates the installation and execution of Frida tools for runtime instrumentation and function hooking. Frida was used to simulate dynamic analysis by attempting to intercept application functions and monitor runtime behavior during Android application security testing.

RUNTIME INSTRUMENTATION USING FRIDA

Frida was installed and configured to simulate runtime function hooking and application behavior interception.

COMMANDS USED

frida --version

frida-trace -U -i "checkPassword" com.example.app

Frida SUMMARY

Frida was used to simulate runtime instrumentation within the Android application environment. Function hooking techniques demonstrated how attackers can intercept application execution, monitor sensitive operations, and potentially bypass authentication controls during runtime analysis.

DROZER IPC TESTING WORKFLOW

Drozer workflows were studied to simulate Android package enumeration and insecure IPC testing activities.

Activities Performed

- Package enumeration
- Exported component analysis

- IPC workflow analysis
- Android application inspection

11.1 CAPSTONE PROJECT EXECUTION SUMMARY

The capstone engagement simulated a complete offensive security assessment against a controlled vulnerable environment using multiple industry-standard penetration testing tools and methodologies. The assessment combined web application exploitation, privilege escalation, network interception, vulnerability scanning, and mobile application analysis into a unified attack workflow.

The engagement began with reconnaissance and vulnerability identification against DVWA and Metasploitable2 targets hosted within the laboratory network. Manual and automated testing techniques were used to validate exploitable weaknesses affecting web applications, APIs, operating systems, and network services.

11.2 ACTIVITIES PERFORMED

WEB APPLICATION EXPLOITATION

- SQL Injection testing using manual payloads and sqlmap
- Reflected XSS exploitation using Burp Suite
- HTTP request interception and manipulation
- Backend database enumeration

EXPLOITATION AND PRIVILEGE ESCALATION

- VSFTPD backdoor exploitation using Metasploit
- Meterpreter session establishment
- Root shell verification using Linux shell commands
- LinPEAS privilege escalation enumeration

NETWORK SECURITY ASSESSMENT

- Wireshark packet capture and protocol analysis
- Responder LLMNR poisoning simulation
- SMB relay simulation
- Ettercap ARP spoofing and MitM testing

MOBILE APPLICATION TESTING

- MobSF static APK analysis
- Android component inspection
- Frida runtime instrumentation simulation

- Dynamic function hooking demonstration

VULNERABILITY ASSESSMENT

- OpenVAS automated vulnerability scanning
- CVSS severity classification
- Risk validation and remediation planning

11.3 CAPSTONE PROJECT FINDINGS

FINDING	SEVERITY	VALIDATION STATUS
SQL Injection	Critical	Confirmed
Reflected XSS	High	Confirmed
VSFTPD Remote Code Execution	Critical	Confirmed
Weak SMB Configuration	High	Confirmed
LLMNR Poisoning Exposure	High	Confirmed
Insecure Android Storage	Medium	Confirmed
Weak Privilege Separation	Critical	Confirmed

11.4 OPENVAS VULNERABILITY ASSESSMENT

OpenVAS vulnerability scanning was performed against the target environment to identify exposed services, outdated software components, insecure configurations, and remotely exploitable vulnerabilities.

Severity Distribution

Severity Level	Findings
Critical	14
High	8
Medium	40
Low	6

OBSERVATION

The OpenVAS assessment identified multiple critical and high-risk vulnerabilities affecting exposed network services and application components. Several findings were associated with insecure service configurations, outdated packages, weak protocol exposure, and insufficient

hardening mechanisms. The scan results validated the presence of vulnerabilities previously identified during manual penetration testing activities.

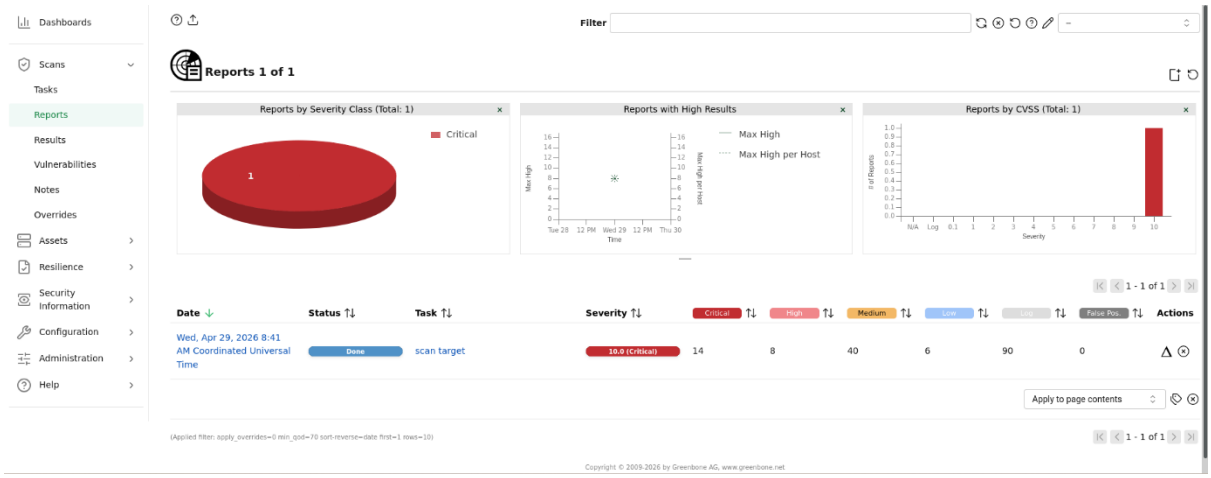

FIGURE 20: *OPENVAS VULNERABILITY ASSESSMENT DASHBOARD*

The OpenVAS dashboard summarizes vulnerability severity distribution and validates the presence of critical security weaknesses identified during manual testing activities.

11.5 OVERALL OBSERVATION

The capstone assessment demonstrated how attackers can combine vulnerabilities across applications, operating systems, APIs, mobile environments, and network protocols to achieve complete compromise scenarios. The engagement validated the importance of secure configuration management, vulnerability remediation, least privilege enforcement, secure coding practices, and continuous penetration testing in reducing long-term cybersecurity risk exposure.

12. FINDINGS SUMMARY

The assessment identified multiple high-risk vulnerabilities across applications, operating systems, network services, and mobile environments. The majority of findings originated from outdated software, insecure configurations, weak input validation, insufficient access control, and exposed services.

KEY FINDINGS INCLUDED:

- VSFTPD Remote Code Execution
- SQL Injection vulnerabilities
- Reflected Cross-Site Scripting

- Weak SMB configurations
- Insecure Android storage mechanisms
- Exposed application components
- Weak privilege separation
- Insecure protocol exposure
- ARP spoofing susceptibility
- API authorization weaknesses

The exploitation workflows demonstrated that attackers could potentially achieve complete system compromise if these weaknesses remain unaddressed.

13. REMEDIATION AND MITIGATION PLAN

The following remediation measures are strongly recommended to reduce attack surface exposure and improve overall security posture:

1. Upgrade vulnerable software and unsupported services immediately.
2. Apply parameterized queries and strict server-side input validation.
3. Implement output encoding and Content Security Policy protections.
4. Restrict unnecessary SUID binaries and excessive permissions.
5. Disable insecure protocols including LLMNR, SMBv1, and Telnet.
6. Enable least privilege enforcement across systems and applications.
7. Deploy centralized logging and continuous monitoring solutions.
8. Harden Android applications using secure storage and runtime protection.
9. Conduct regular vulnerability scanning and penetration testing.
10. Perform periodic OpenVAS rescans to validate remediation effectiveness.

14. EXECUTIVE SUMMARY FOR STAKEHOLDERS

A controlled cybersecurity assessment was performed to evaluate the security posture of the testing environment and identify weaknesses capable of exposing systems, applications, and sensitive information to unauthorized access. The engagement demonstrated how outdated software, insecure application behavior, weak network configurations, and insufficient access controls can increase operational cybersecurity risk.

Several vulnerabilities were successfully identified and validated during testing, including weaknesses affecting web applications, APIs, operating systems, mobile applications, and network communication protocols. Exploitation activities demonstrated that attackers could

potentially gain unauthorized access, intercept traffic, manipulate application functionality, and escalate privileges within vulnerable environments.

Recommendations were provided to improve overall security resilience, including software patching, secure coding practices, access control hardening, network segmentation, continuous monitoring, and vulnerability management improvements.

The engagement highlights the importance of proactive penetration testing and continuous security validation in reducing attack surface exposure and strengthening organizational cyber defense capabilities.

15. CONCLUSION

The Week 4 Advanced VAPT assessment successfully demonstrated a complete offensive security workflow covering reconnaissance, exploitation, privilege escalation, network interception, mobile application analysis, persistence simulation, and remediation planning within a controlled laboratory environment.

Multiple vulnerabilities including Remote Code Execution, SQL Injection, Cross-Site Scripting, insecure SMB behavior, weak privilege separation, and insecure mobile configurations were successfully identified and validated using industry-standard offensive security tools.

The assessment demonstrated how chained vulnerabilities and insecure configurations can significantly increase organizational exposure to unauthorized access, data disclosure, privilege escalation, and system compromise.

The engagement further emphasized the importance of secure coding practices, patch management, least privilege enforcement, network hardening, continuous monitoring, and regular penetration testing in maintaining long-term cybersecurity resilience.

16. REFERENCES

- OWASP Web Security Testing Guide
- OWASP API Security Top 10
- OWASP Mobile Security Testing Guide
- Metasploit Framework Documentation
- OpenVAS Documentation
- Nmap Reference Guide
- sqlmap Documentation

- HackTricks Privilege Escalation Guide
- PTES Technical Guidelines
- Wireshark Documentation
- MobSF Documentation
- Frida Documentation
- Exploit-DB Reference Repository
- TCM Security Practical Pentesting Resources
- SANS Security Research Materials